15-Nio2Endpoint组件：Tomcat如何实现异步IO？

我在专栏上一期里提到了5种I/O模型，相应的，Java提供了BIO、NIO和NIO.2这些API来实现这些I/O模型。BIO是我们最熟悉的同步阻塞，NIO是同步非阻塞，那NIO.2又是什么呢？NIO已经足够好了，为什么还要NIO.2呢？

NIO和NIO.2最大的区别是，一个是同步一个是异步。我在上期提到过，异步最大的特点是，应用程序不需要自己去**触发**数据从内核空间到用户空间的**拷贝**。

为什么是应用程序去“触发”数据的拷贝，而不是直接从内核拷贝数据呢？这是因为应用程序是不能访问内核空间的，因此数据拷贝肯定是由内核来做，关键是谁来触发这个动作。

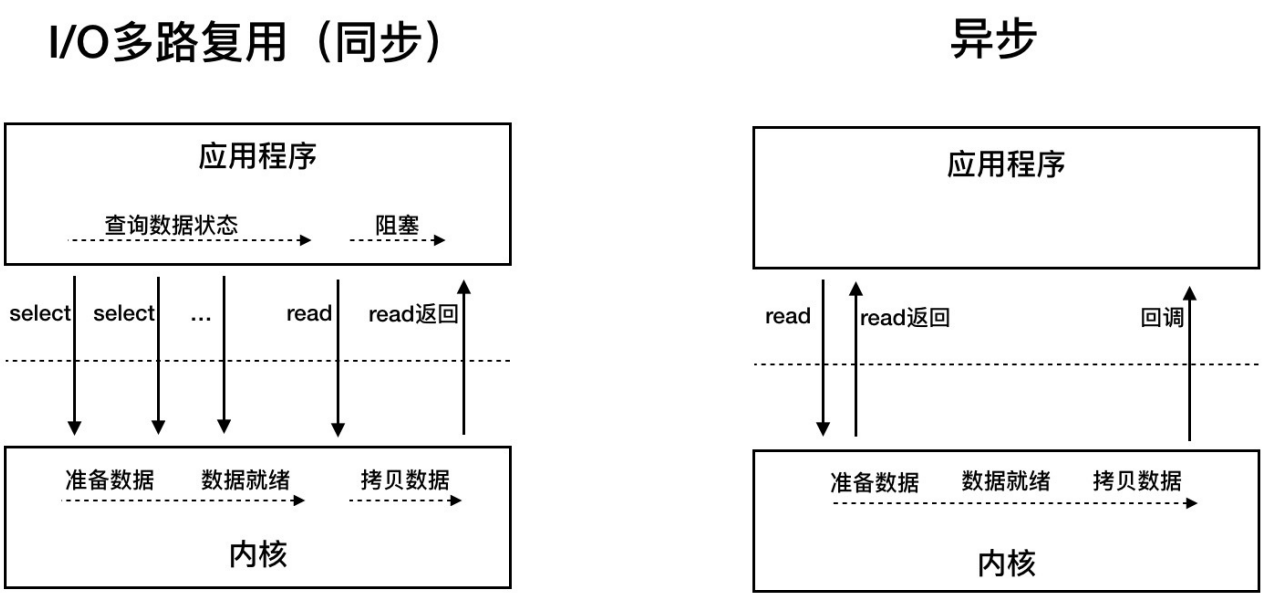
是内核主动将数据拷贝到用户空间并通知应用程序。还是等待应用程序通过Selector来查询，当数据就绪后，应用程序再发起一个read调用，这时内核再把数据从内核空间拷贝到用户空间。

需要注意的是，数据从内核空间拷贝到用户空间这段时间，应用程序还是阻塞的。所以你会看到异步的效率是高于同步的，因为异步模式下应用程序始终不会被阻塞。下面我以网络数据读取为例，来说明异步模式的工作过程。

首先，应用程序在调用read API的同时告诉内核两件事情：数据准备好了以后拷贝到哪个Buffer，以及调用哪个回调函数去处理这些数据。

之后，内核接到这个read指令后，等待网卡数据到达，数据到了后，产生硬件中断，内核在中断程序里把数据从网卡拷贝到内核空间，接着做TCP/IP协议层面的数据解包和重组，再把数据拷贝到应用程序指定的Buffer，最后调用应用程序指定的回调函数。

你可能通过下面这张图来回顾一下同步与异步的区别：



我们可以看到在异步模式下，应用程序当了“甩手掌柜”，内核则忙前忙后，但最大限度提高了I/O通信的效率。Windows的IOCP和Linux内核2.6的AIO都提供了异步I/O的支持，Java的NIO.2 API就是对操作系统异步I/O API的封装。

Java NIO.2回顾

仅供个人学习 请勿传播 用户标记 mannix小食鞋

今天我们会重点关注Tomcat是如何实现异步I/O模型的，但在这之前，我们先来简单回顾下如何用Java的NIO.2 API来编写一个服务端程序。

```
public class Nio2Server {

   void listen(){
      //1.创建一个线程池
      ExecutorService es = Executors.newCachedThreadPool();

      //2.创建异步通道群组
      AsynchronousChannelGroup tg = AsynchronousChannelGroup.withCachedThreadPool(es, 1);

      //3.创建服务端异步通道
      AsynchronousServerSocketChannel assc = AsynchronousServerSocketChannel.open(tg);

      //4.绑定监听端口
      assc.bind(new InetSocketAddress(8080));

      //5. 监听连接，传入回调类处理连接请求
      assc.accept(this, new AcceptHandler());
   }
}
```

上面的代码主要做了5件事情：

1. 创建一个线程池，这个线程池用来执行来自内核的回调请求。
2. 创建一个AsynchronousChannelGroup，并绑定一个线程池。
3. 创建AsynchronousServerSocketChannel，并绑定到AsynchronousChannelGroup。
4. 绑定一个监听端口。
5. 调用accept方法开始监听连接请求，同时传入一个回调类去处理连接请求。请你注意，accept方法的第一个参数是this对象，就是Nio2Server对象本身，我在下文还会讲为什么要传入这个参数。

你可能会问，为什么需要创建一个线程池呢？其实在异步I/O模型里，应用程序不知道数据在什么时候到达，因此向内核注册回调函数，当数据到达时，内核就会调用这个回调函数。同时为了提高处理速度，会提供一个线程池给内核使用，这样不会耽误内核线程的工作，内核只需要把工作交给线程池就立即返回了。

我们再来看看处理连接的回调类AcceptHandler是什么样的。

```
//AcceptHandler类实现了CompletionHandler接口的completed方法。它还有两个模板参数，第一个是异步通道，第二个就是Nio2Serve
public class AcceptHandler implements CompletionHandler<AsynchronousSocketChannel, Nio2Server> {

   //具体处理连接请求的就是completed方法，它有两个参数：第一个是异步通道，第二个就是上面传入的NioServer对象
   @Override
   public void completed(AsynchronousSocketChannel asc, Nio2Server attachment) {
      //调用accept方法继续接收其他客户端的请求
      attachment.assc.accept(attachment, this);

      //1. 先分配好Buffer，告诉内核，数据拷贝到哪里去
      ByteBuffer buf = ByteBuffer.allocate(1024);

      //2. 调用read函数读取数据，除了把buf作为参数传入，还传入读回调类
      channel.read(buf, buf, new ReadHandler(asc));
```

```
}
```

我们看到它实现了CompletionHandler接口，下面我们先来看看CompletionHandler接口的定义。

```
public interface CompletionHandler<V,A> {

    void completed(V result, A attachment);

    void failed(Throwable exc, A attachment);
}
```

CompletionHandler接口有两个模板参数V和A，分别表示I/O调用的返回值和附件类。比如accept的返回值就是AsynchronousSocketChannel，而附件类由用户自己决定，在accept的调用中，我们传入了一个Nio2Server。因此AcceptHandler带有了两个模板参数：AsynchronousSocketChannel和Nio2Server。

CompletionHandler有两个方法：completed和failed，分别在I/O操作成功和失败时调用。completed方法有两个参数，其实就是前面说的两个模板参数。也就是说，Java的NIO.2在调用回调方法时，会把返回值和附件类当作参数传给NIO.2的使用者。

下面我们再来看看处理读的回调类ReadHandler长什么样子。

```
public class ReadHandler implements CompletionHandler<Integer, ByteBuffer> {
    //读取到消息后的处理
    @Override
    public void completed(Integer result, ByteBuffer attachment) {
        //attachment就是数据，调用flip操作，其实就是把读的位置移动最前面
        attachment.flip();
        //读取数据
        ...
    }

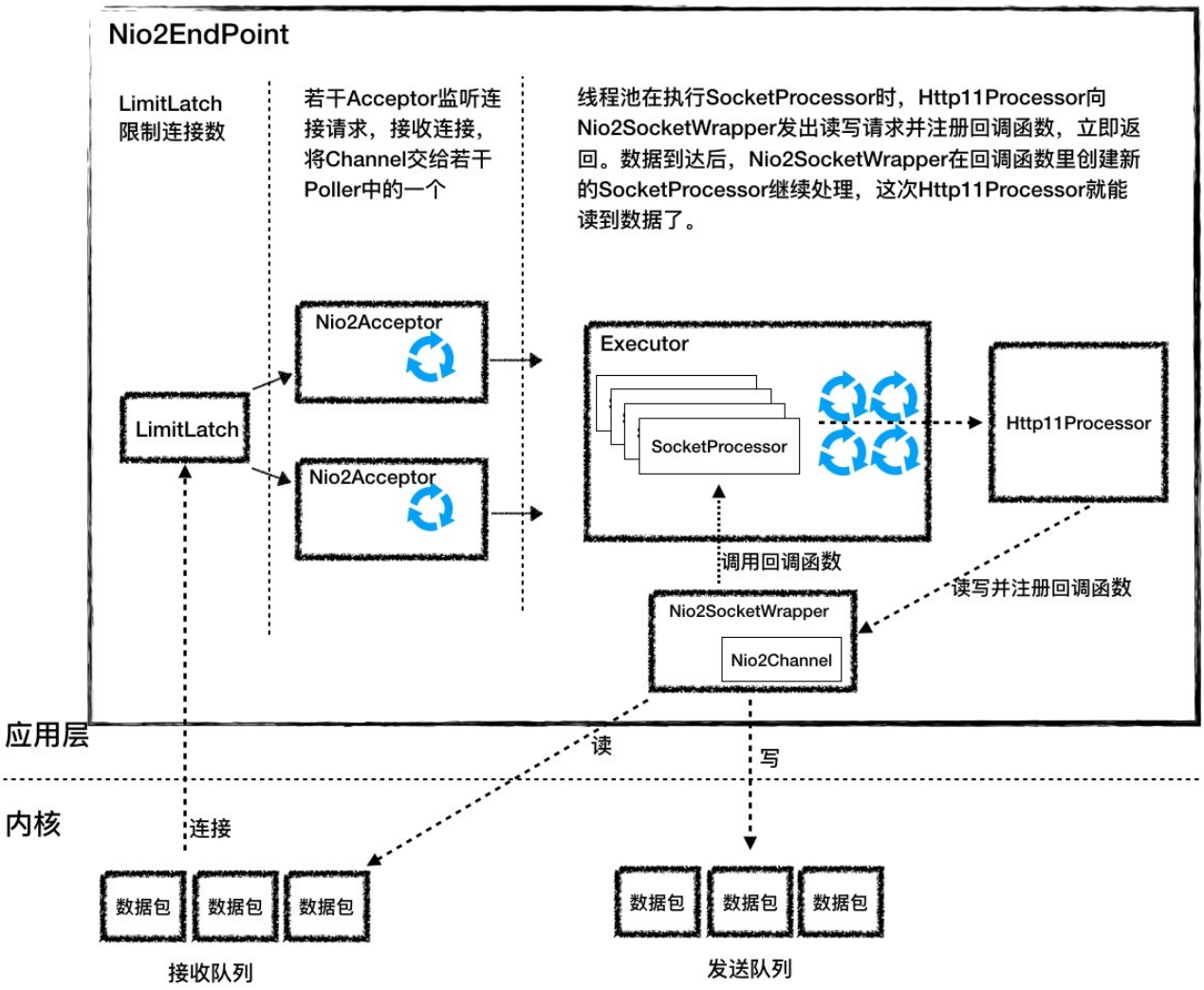
    void failed(Throwable exc, A attachment){
        ...
    }
}
```

read调用的返回值是一个整型数，所以我们回调方法里的第一个参数就是一个整型，表示有多少数据被读取到了Buffer中。第二个参数是一个ByteBuffer，这是因为我们在调用read方法时，把用来存放数据的ByteBuffer当作附件类传进去了，所以在回调方法里，有ByteBuffer类型的参数，我们直接从这个ByteBuffer里获取数据。

Nio2Endpoint

掌握了Java NIO.2 API的使用以及服务端程序的工作原理之后，再来理解Tomcat的异步I/O实现就不难了。我们先通过一张图来看看Nio2Endpoint有哪些组件。

仅供个人学习 请勿传播 用户标记 mannix小查理



从图上看，总体工作流程跟NioEndpoint是相似的。

LimitLatch是连接控制器，它负责控制最大连接数。

Nio2Acceptor扩展了Acceptor，用异步I/O的方式来接收连接，跑在一个单独的线程里，也是一个线程组。Nio2Acceptor接收新的连接后，得到一个AsynchronousSocketChannel，Nio2Acceptor把AsynchronousSocketChannel封装成一个Nio2SocketWrapper，并创建一个SocketProcessor任务类交给线程池处理，并且SocketProcessor持有Nio2SocketWrapper对象。

Executor在执行SocketProcessor时，SocketProcessor的run方法会调用Http11Processor来处理请求，Http11Processor会通过Nio2SocketWrapper读取和解析请求数据，请求经过容器处理后，再把响应通过Nio2SocketWrapper写出。

需要你注意Nio2Endpoint跟NioEndpoint的一个明显不同点是，**Nio2Endpoint中没有Poller组件，也就是没有Selector。这是为什么呢？因为在异步I/O模式下，Selector的工作交给内核来做了。**

接下来我详细介绍一下Nio2Endpoint各组件的设计。

Nio2Acceptor

和NioEndpint一样，Nio2Endpoint的基本思路是用LimitLatch组件来控制连接数，但是Nio2Acceptor的监听连接的过程不是在一个死循环里不断的调accept方法，而是通过回调函数来完成的。我们来看看它的连接监听方法：

```
serverSock.accept(null, this);
```

其实就是调用了accept方法，注意它的第二个参数是this，表明Nio2Acceptor自己就是处理连接的回调类，因此Nio2Acceptor实现了CompletionHandler接口。那么它是如何实现CompletionHandler接口的呢？

```
protected class Nio2Acceptor extends Acceptor<AsynchronousSocketChannel>
    implements CompletionHandler<AsynchronousSocketChannel, Void> {

@Override
public void completed(AsynchronousSocketChannel socket,
        Void attachment) {

    if (isRunning() && !isPaused()) {
        if (getMaxConnections() == -1) {
            //如果没有连接限制，继续接收新的连接
            serverSock.accept(null, this);
        } else {
            //如果有连接限制，就在线程池里跑Run方法，Run方法会检查连接数
            getExecutor().execute(this);
        }
        //处理请求
        if (!setSocketOptions(socket)) {
            closeSocket(socket);
        }
    }
}
```

可以看到CompletionHandler的两个模板参数分别是AsynchronousServerSocketChannel和Void，我在前面说过第一个参数就是accept方法的返回值，第二个参数是附件类，由用户自己决定，这里为Void。completed方法的处理逻辑比较简单：

- 如果没有连接限制，继续在本线程中调用accept方法接收新的连接。
- 如果有连接限制，就在线程池里跑run方法去接收新的连接。那为什么要跑run方法呢，因为在run方法里会检查连接数，当连接达到最大数时，线程可能会被LimitLatch阻塞。为什么要放在线程池里跑呢？这是因为如果放在当前线程里执行，completed方法可能被阻塞，会导致这个回调方法一直不返回。

接着completed方法会调用setSocketOptions方法，在这个方法里，会创建Nio2SocketWrapper和SocketProcessor，并交给线程池处理。

Nio2SocketWrapper

Nio2SocketWrapper的主要作用是封装Channel，并提供接口给Http11Processor读写数据。讲到这里你是不是有个疑问：Http11Processor是不能阻塞等待数据的，按照异步I/O的套路，Http11Processor在调用Nio2SocketWrapper的read方法时需要注册回调类，read调用会立即返回，问题是立即返回后Http11Processor还没有读到数据， 怎么办呢？这个请求的处理不就失败了吗？

为了解决这个问题，Http11Processor是通过2次read调用来完成数据读取操作的。

仅供个人学习 请勿传播 用户标记 mannix小童鞋

- 第一次read调用：连接刚刚建立好后，Acceptor创建SocketProcessor任务类交给线程池去处理，Http11Processor在处理请求的过程中，会调用Nio2SocketWrapper的read方法发出第一次读请求，同时注册了回调类readCompletionHandler，因为数据没读到，Http11Processor把当前的Nio2SocketWrapper标记为数据不完整。**接着SocketProcessor线程被回收，Http11Processor并没有阻塞等待数据**。这里请注意，Http11Processor维护了一个Nio2SocketWrapper列表，也就是维护了连接的状态。
- 第二次read调用：当数据到达后，内核已经把数据拷贝到Http11Processor指定的Buffer里，同时回调类readCompletionHandler被调用，在这个回调处理方法里会**重新创建一个新的SocketProcessor任务来继续处理这个连接**，而这个新的SocketProcessor任务类持有原来那个Nio2SocketWrapper，这一次Http11Processor可以通过Nio2SocketWrapper读取数据了，因为数据已经到了应用层的Buffer。

这个回调类readCompletionHandler的源码如下，最关键的一点是，**Nio2SocketWrapper是作为附件类来传递的**，这样在回调函数里能拿到所有的上下文。

```
this.readCompletionHandler = new CompletionHandler<Integer, SocketWrapperBase<Nio2Channel>>() {
    public void completed(Integer nBytes, SocketWrapperBase<Nio2Channel> attachment) {
        ...
        //通过附件类SocketWrapper拿到所有的上下文
        Nio2SocketWrapper.this.getEndpoint().processSocket(attachment, SocketEvent.OPEN_READ, false);
    }

    public void failed(Throwable exc, SocketWrapperBase<Nio2Channel> attachment) {
        ...
    }
}
```

本期精华

在异步I/O模型里，内核做了很多事情，它把数据准备好，并拷贝到用户空间，再通知应用程序去处理，也就是调用应用程序注册的回调函数。Java在操作系统 异步IO API的基础上进行了封装，提供了Java NIO.2 API，而Tomcat的异步I/O模型就是基于Java NIO.2 实现的。

由于NIO和NIO.2的API接口和使用方法完全不同，可以想象一个系统中如果已经支持同步I/O，要再支持异步I/O，改动是比较大的，很有可能不得不重新设计组件之间的接口。但是Tomcat通过充分的抽象，比如SocketWrapper对Channel的封装，再加上Http11Processor的两次read调用，巧妙地解决了这个问题，使得协议处理器Http11Processor和I/O通信处理器Endpoint之间的接口保持不变。

课后思考

我在文章开头介绍Java NIO.2的使用时，提到过要创建一个线程池来处理异步I/O的回调，那么这个线程池跟Tomcat的工作线程池Executor是同一个吗？如果不是，它们有什么关系？

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。



精选留言：

- yungoo 2019-06-13 09:25:22
  我所看的tomcat 8.5的代码跟专栏所讲已经有些不一致了。已经没有Nio2Acceptor了，accept获取连接用的是Future aceept()。 [1赞]

  作者回复2019-06-13 15:15:12
  我使用的是最新版的代码：
  https://github.com/apache/tomcat/blob/master/java/org/apache/tomcat/util/net/Nio2Endpoint.java#L382

- nimil 2019-06-13 21:10:59
  问下老师，这个Tomcat这个IO模型是将数据拷贝了两次么，还是有做特殊优化

  作者回复2019-06-13 22:07:44
  数据没有拷贝两次，第一次read调用是读不到数据的，因为这个时候数据还没应用层的Buffer，只是注册一个回调函数，当内核将数据拷贝到了应用层Buffer，调用回调函数，在回调函数里，HttpProccessor再发起一次read，read方法首先会检查数据是不是已经到了Buffer，如果是，直接读取Buffer返回，这一次并没有真正向内核发起read调用。

- W.T 2019-06-13 20:50:41
  李老师实力派！👍

- 木偶人King 2019-06-13 17:36:43
  前边看的比较顺畅。 今天的没太看懂。顿生挫败感。

  加油加油。 只能多看几遍了

- 半斤八两 2019-06-13 15:33:28
  老师图片上Acceptor的行为还是交给poller，是不是有问题啊……还是我理解错了……

  作者回复2019-06-13 22:27:13

仅供个人学习 请勿传播 用户标记 mannix小童鞋

这一篇的图片上没有poller吧

- 2019-06-13 10:02:08
  @WL Linux 的内存映射了解下

- WL 2019-06-13 08:57:43
  老师关于IO模型内存我有两个问题：
  1. 配内存的时候，是不是因为堆内存会受到GC的影响导致地址变化，所以不能直接使用不能使用堆内存， 如果使用堆内存的话也需要先指向一个固定的堆外内存，所以使用堆外内存就可以避免GC对内存地址的影响。
  2. 是不是IO在读数据的时候经过两次数据拷贝，从网卡到内核态，从内核态到用户态，这两次数据拷贝有什么方法可以减少1次吗？

  作者回复2019-06-13 15:15:53
  好问题，下一篇就会解释这个问题。